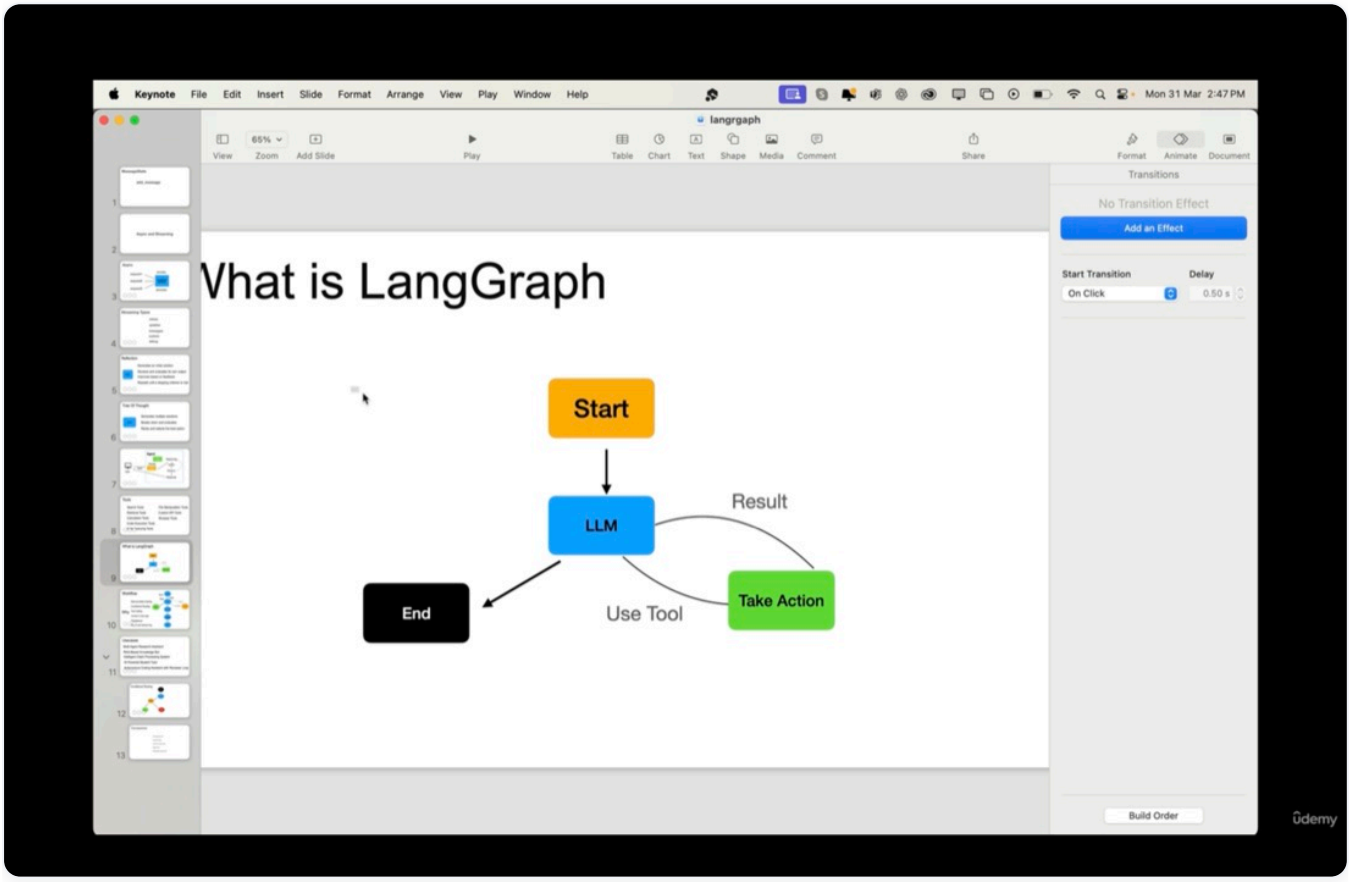
LANGGRAPH

langgraphdemo

Source: Notes.md

1. When we create AI agents using LangChain, these AI agents are LLMs with access to external tools. They operate in a loop, acting as a reasoning engine that decides which tool they should use at each step to get the given work done.



LangChain

But as this agentic workflow becomes complex, this linear flow of LangChain is not good enough — that's where LangGraph comes into play.

2. LangGraph, built on top of LangChain, allows us to create graph-based workflows to orchestrate the behavior of AI agents.

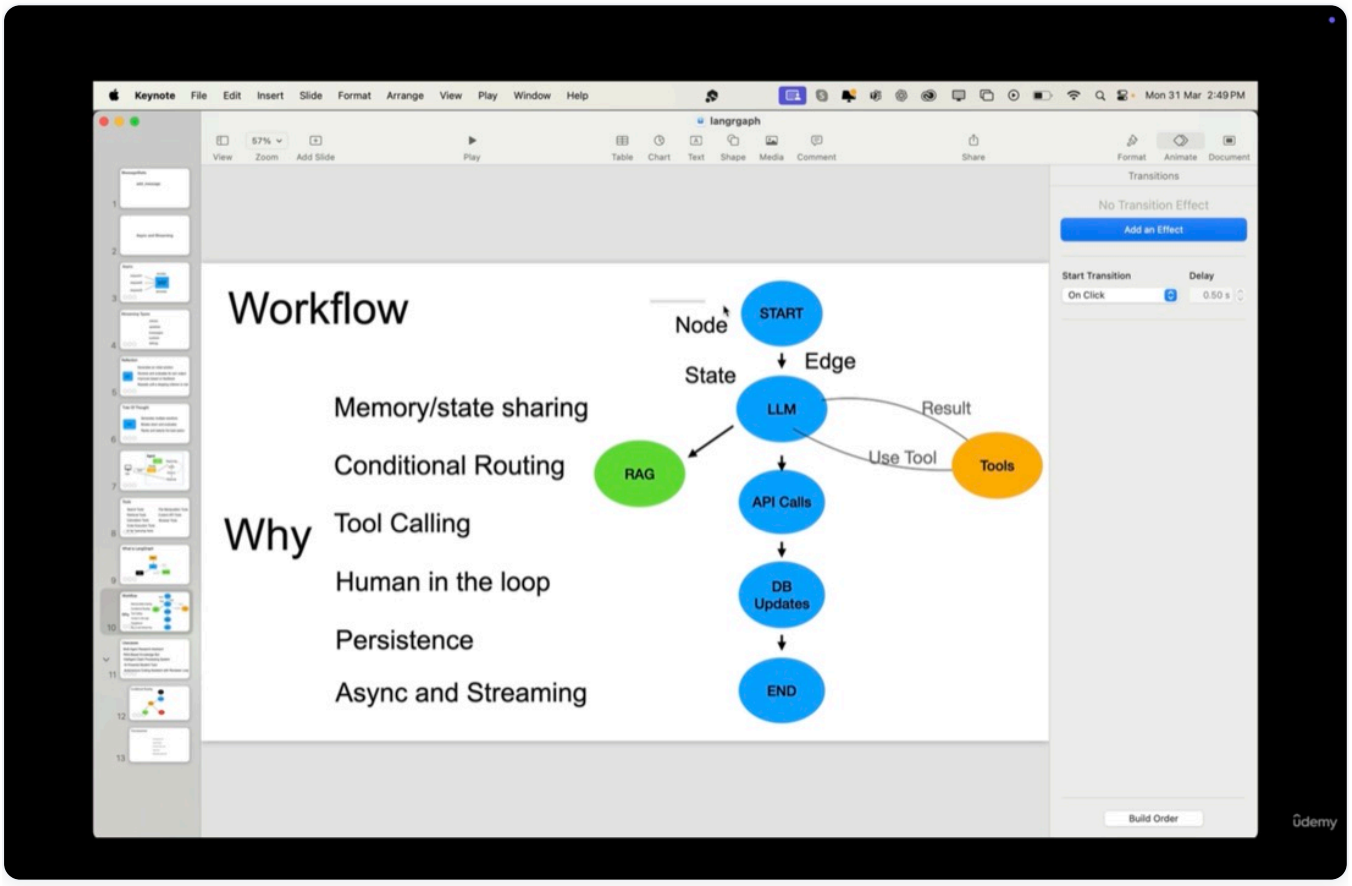
A LangGraph contains 3 important components:

- Nodes: A node represents a specific task ranging from simple processing logic to a complex self-contained agent. Each node can itself be an agent of its own.

- Edges: Each node can be connected with an edge. An edge represents the transition between different nodes in the workflow or graph.
- State: It represents the state of the workflow, and as the workflow happens, the state gets updated at each node.

LangGraph comes with the following in-built features:

- Memory/state sharing
- Conditional Routing
- Tool Calling
- Human in the loop
- Persistence
- Async and Streaming



LangGraph

3. Hello World LangGraph workflow:

Every LangGraph workflow starts with the state definition.

please check the code in `hello_world.py` in `1.basics` folder and see the comments.

4. For setup of this tutorial, go to `1.langgraphdemo` folder and run `python3 -m venv .venv` and then `source .venv/bin/activate` and `pip3 install -r requirements.txt`.
5. Now go to `1.basics` folder and run `python3 1.basics/hello_world.py` and see the output as:

```
 ankit@MacBook-Air 💻  …/AI-ML-DS/26.LangGraph/1.langgraphdemo on  main [ *1 ?102  ] 🐍
(.venv) 🐍  v3.14.2
└─ python3 1.basics/hello_world.py
/Users/ankit/Workspace/Projects/ankit-github/AI-ML-
DS/26.LangGraph/1.langgraphdemo/.venv/lib/python3.14/site-
packages/langgraph/cache/base/__init__.py:8: LangChainPendingDeprecationWarning: The
default value of `allowed_objects` will change in a future version. Pass an explicit value
(e.g., allowed_objects='messages' or allowed_objects='core') to suppress this warning.
  from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer
Hello Node: Ankit
Bye Node: Hello Ankit
Final Output: {'message': 'Bye Hello Ankit'}
```

6. Graph must have an entrypoint: add at least one edge from START to another node.

You can just delete `graph.add_edge(START, "hello") # we add an edge from START to hello node` from `hello_world.py` and run the script `python3 1.basics/hello_world1.py` and see the error.

To resolve this error, we can also add `graph.set_entry_point("hello") # we set the entry point of the graph to hello node` in the code and then run the script `python3 1.basics/hello_world2.py` and see the output as:

7. Now, to see the workflow as an image, run the script as `python3 1.basics/hello_world3.py` and see the image.
8. We use the `Pydantic` library to validate the state of the graph.

To restrict the length of the message, we can use the `Field` method.

Run the script as `python3 1.basics/hello_world4.py` and see the output.

9. You can make the “id” field optional by importing `Optional` from the `typing` library.

Run the script as `python3 1.basics/hello_world5.py` and see the output.

10. When we launch a LangGraph workflow using the `invoke` method, everything happens in a synchronous fashion, that is, the main thread in which the work happens is blocked until all the nodes are done with their work. When any new requests come, they have to wait until this blocked thread completes. That is where asynchronous invocation comes into the picture, and we use the `ainvoke` method to do everything in an asynchronous manner.

The asynchronous method is simple. Use the `async` keyword before any node functions, and then we use the `runnable.ainvoke()` method in the async context of the main thread to run the workflow.

Run the script as `python3 2.asyncandstreaming/async_demo.py` and see the output with delay of 1 second.

11. Langgraph also supports streaming.

Streaming Types: values, updates, messages, custom, debug.

Run the script as `python3 2.asyncandstreaming/streaming_demo.py` and see the output.

12. Conditional Routing:

Here, we take the example of a customer service use case. Here, the user sends 2 things: the request and the priority.

Now, we can run the script as `python3 3.conditional_routing/customer_service.py` and see the output.

13. Reducers:

Using reducers, we can override messages.

Run the script as `python3 4.reducers/reducer_demo.py` and see the output, where the `add` function works as a reducer and will add the discount.

For messages state, run the script as `python3 4.reducers/reducers_messagesstate_demo.py` and see the output.

14. Tool Calling:

Run the script as `python3 5.tool_calling/toolcalling_demo.py` and see the output.

Run the script as `python3 5.tool_calling/toolnode_manual.py` and see the output.

Run the script as `python3 5.tool_calling/toolnode_auto.py` and see the output.

15. Agentic RAG:

Run the script as `python3 6.agentic_rag/rag_demo.py` and see the output.

16. Human in the loop (HITL):

HITL is a pattern where an AI agent pauses its execution to seek human validation, input and decision making etc. for continuing the workflow.

HITL can be applied for various use cases like:

A. Accept/Reject:

- Report to stakeholders: Before the report is sent out, it is reviewed by someone in the middle, and it can be either accepted or rejected. If it is accepted, then the report is sent out by the agent, and if it is rejected, then it will be summarized again by the agent.

B. Review Tool Calls:

This is where the agent comes up with various options which need to be reviewed by a human before the agent passes it to the next node.

C. Fallback when agent is uncertain:

A chatbot is a good example here. If the chatbot is not confident to understand the user's query from its knowledge base, then it asks the user to give a certain answer to proceed further.

D. Review and Edit State:

This is where human can review and edit the state of the workflow to proceed with next node.

Here, we see the code and test generation use case for HITL. LLM will generate the code for us and then human will review and then accordingly test cases will be generated.

Here we use `interrupt` and `command` functions from langgraph.

Run the script as `python3 7.hitl/code_generator.py` and see the output.

Run the script as `python3 7.hitl/test_generator1.py` and see the output.

17. Long Term Memory (using Postgres):

Earlier we used memory with `checkpointer` using `MemorySaver()` as workflows and it is in-memory only. Once the program execution is done and since it is in-memory so after the program execution is done then the memory is lost.

For persistence, we use permanent memory.

We use the Postgres database with `PostgresSaver()` function.

So, first download and install the Postgres database in your machine with username and password.

For my system, pgadmin 4 v9.9 with `PostgresSQL 18` is installed. So, open it and then create a new database named `langgraph_memory`. Make sure postgres is up and running.

Here, we use postgres as our permanent state store.

Now, run the script as `python3 8.persistent_memory/permanent_memory.py` and see the output.

Also, check the created tables in postgres database where states are stored in the tables.

18. Subgraphs:

A subgraph is a graph or workflow of its own which can be used across multiple other graphs.

It is like encapsulation applied to langgraph. The advantage of subgraphs is: it makes the complex multi-agentic systems simple.

They offer modularity, meaning we can break down complex workflows into smaller, simpler workflows which can be worked on by different teams. It is easy to develop and maintain them.

It can also provide reusability, testability, clean and better design etc.

There are 2 ways to use in a graph:

We can use a subgraph as a node in the big graph if they have common states/keys (`patient_appointment.py`), otherwise we have to invoke the subgraph from the parent node (`rag_demo.py`).

Run the script as `python3 9.subgraphs/patient_appointment.py` and see the output.

Run the script as `python3 9.subgraphs/rag_demo.py` and see the output.

19. Now, based on the acquired knowledge of LangGraph, we will create an app where we give the patient id, treatment code, and claim details, and the bot will say whether the claim should be approved or not, and if more info is required, then human-in-the-loop or human review comes in.

First, go to pgadmin and create a new database with name “claims_db” then expand this database and go to schemas–>tables and then click on query tools and then copy and paste the content of claims.sql file and execute it.

Run the code as `chainlit run app.py --port 8001` from the `10.usecase` folder and see the output after putting the patient id, treatment code and claim details from `test_data.txt` file.

For FASTAPI implementation:

run the code as `uvicorn claim_processing_api:app --reload --port 8001` from the `10.usecase` folder and see the output in `http://127.0.0.1:8001/docs` after putting the patient id, treatment code and claim details from `test_data.txt` file.

20. Agentic Patterns:

These are ready to use blueprints for a problem which we are trying to solve. It enables scalability, modularity and adaptability.

There can be many agentic patterns like:

A. Reflection: Here agent reviews and improves its own response before the finalization.

The Reflection pattern in an AI agent is a self-improvement mechanism where an agent generates an initial solution for the given problem. It also reviews and evaluates its own output. It also improves based on the feedback. It repeats this process until the stopping criteria are met.

The advantage of the Reflection pattern is that it enables an AI agent to learn from its own mistakes and refine the output iteratively.

For use case, run the script as `python3 11.patterns/reflection_agent.py` and see the output. Here, we have code generation agent.

B. Tree of Thoughts: Here, agent explores multiple reasoning paths and selects the best one.

The Tree of Thought pattern is an AI reasoning pattern where agent generates multiple possible solutions for a given problem. Then it breaks down and evaluates each of those solutions separately and then it ranks and selects the best solution.

For the use case, run the script as `python3 11.patterns/tree_of_thought.py` and see the output. Here we have business expansion strategy agent.

C. Parallel Execution: Here agent performs multiple tasks simultaneously to increase the speed and efficiency.

LangGraph has parallel processing in-built.

Run the script as `python3 11.patterns/parallel_processing.py` and see the output.

21.